

Prompt Examples

Repo Evolution

I want to add marimo notebook support to the PyMC progress bar system while keeping it fully backward compatible with existing terminal and Jupyter environments. Currently the progress bar is implemented in a single `pymc/progress_bar.py` file using the Rich library with a `CustomProgress` class that extends `Rich Progress` for terminal rendering. When sampling functions run inside a marimo notebook the progress bars should render as styled HTML tables that update in-place within the cell output instead of using Rich terminal rendering which does not display correctly in marimo. The system should automatically detect the execution environment and select the appropriate rendering without requiring any user configuration. In standard Python terminals and Jupyter notebooks the existing progress bar behavior should be preserved exactly as it currently functions.

The `pm.sample` function has a `progressbar` parameter that currently supports multiple display modes through a `ProgressBarType` and this should continue to work. The `progressbar` parameter should accept `True` for the default mode showing per-chain progress with statistics or `False` to disable progress display entirely or combined for a single aggregated progress bar across all chains or `split` for separate per-chain bars without detailed statistics or `combined+stats` and `split+stats` for explicit control over statistics display. Progress bars should visually indicate sampling failures by changing color when divergences or other issues are detected. The display should show current draw count and total draws and completion percentage and sampling speed and elapsed time. The HTML output for marimo should have a clean modern appearance and should support both light and dark color schemes. Users should be able to provide custom CSS styling through a `progressbar_css` parameter if they want to customize the appearance. The `sample_posterior_predictive` function should also display progress appropriately based on the environment. The implementation should create a `pymc/progress_bar/` submodule that refactors the existing single-file progress bar into a modular architecture with separate backends for Rich terminal rendering and marimo HTML rendering. The `pymc/progress_bar/_init_.py` should export `ProgressBarManager` and `ProgressBarType` and `create_simple_progress` and `default_progress_theme` and `CustomProgress` and `abbreviate_stat_name` and `compute_draw_speed` and `format_time`. The old `pymc/progress_bar.py` file should be deleted after refactoring. The `pymc/sampling/mcmc.py` should be modified to use `ProgressBarManager` and `pymc/sampling/forward.py` should be modified to use `create_simple_progress`. Environment detection should happen automatically using the `in_marimo_notebook` function. All existing progressbar parameter values and behavior should remain compatible.

Path: `pymc/progress_bar/progress.py`

Name: `ProgressBackend`

Type: Protocol

Input: None

Output: None

Annotations: Protocol from typing

Description: Defines the contract for progress bar rendering backends

Path: `pymc/progress_bar/progress.py`

Name: `ProgressBackend.is_enabled`

Type: property

Input: self

Output: bool

Description: Returns whether the progress bar is enabled

Path: `pymc/progress_bar/progress.py`

Name: `ProgressBackend._enter_`

Type: method

Input: self

Output: `ProgressBackend`

Description: Context manager entry point

Path: pymc/progress_bar/progress.py

Name: ProgressBackend._exit_

Type: method

Input: self, exc_type, exc_val, exc_tb

Output: None

Description: Context manager exit point

Path: pymc/progress_bar/progress.py

Name: ProgressBackend.update

Type: method

Input: self, chain_idx int, draw int, failing bool, stats dict, is_last bool

Output: None

Description: Updates progress for a specific chain

Path: pymc/progress_bar/progress.py

Name: ProgressBarManager

Type: class

Input: None

Output: None

Description: Manages progress bars during sampling and automatically detects execution environment

Path: pymc/progress_bar/progress.py

Name: ProgressBarManager._init_

Type: method

Input: self, step_method, chains int, draws int, tune int, progressbar, progressbar_theme, progressbar_css

Output: None

Description: Initializes the progress bar manager

Path: pymc/progress_bar/progress.py

Name: ProgressBarManager._is_marimo

Type: property

Input: self

Output: bool

Description: Returns True if running in marimo notebook

Path: pymc/progress_bar/progress.py

Name: ProgressBarManager._enter_

Type: method

Input: self

Output: ProgressBarManager

Description: Context manager entry

Path: pymc/progress_bar/progress.py

Name: ProgressBarManager._exit_

Type: method

Input: self, exc_type, exc_val, exc_tb

Output: None

Description: Context manager exit

Path: pymc/progress_bar/progress.py

Name: ProgressBarManager._extract_stats

Type: method

Input: self, stats

Output: tuple of bool and dict
Description: Extracts failing status and statistics

Path: pymc/progress_bar/progress.py
Name: ProgressBarManager.update
Type: method
Input: self, chain_idx int, is_last bool, draw int, tuning bool, stats
Output: None
Description: Updates progress bar for a specific chain

Path: pymc/progress_bar/progress.py
Name: create_simple_progress
Type: function
Input: description str, total int, progressbar bool, progressbar_theme, progressbar_css
Output: CustomProgress or MarimoSimpleProgress
Description: Factory function that creates appropriate simple progress bar

Path: pymc/progress_bar/rich_progress.py
Name: default_progress_theme
Type: Theme constant
Input: None
Output: Theme
Description: Default Rich theme with blue color scheme

Path: pymc/progress_bar/rich_progress.py
Name: CustomProgress
Type: class
Bases: Progress
Input: None
Output: None
Description: A child of Progress that allows disabling progress bars

Path: pymc/progress_bar/rich_progress.py
Name: CustomProgress._init_
Type: method
Input: self, args, disable bool, include_headers bool, kwargs
Output: None
Description: Initializes with optional disable flag

Path: pymc/progress_bar/rich_progress.py
Name: CustomProgress._enter_
Type: method
Input: self
Output: CustomProgress
Description: Context manager entry

Path: pymc/progress_bar/rich_progress.py
Name: CustomProgress._exit_
Type: method
Input: self, exc_type, exc_val, exc_tb
Output: None
Description: Context manager exit

Path: pymc/progress_bar/rich_progress.py
Name: CustomProgress.add_task

Type: method
Input: self, args, kwargs
Output: TaskID or None
Description: Adds a task to the progress bar

Path: pymc/progress_bar/rich_progress.py
Name: CustomProgress.advance

Type: method
Input: self, task_id, advance
Output: None
Description: Advances a task by specified amount

Path: pymc/progress_bar/rich_progress.py
Name: CustomProgress.update

Type: method
Input: self, task_id, kwargs
Output: None
Description: Updates task fields

Path: pymc/progress_bar/rich_progress.py
Name: CustomProgress.make_tasks_table

Type: method
Input: self, tasks Iterable of Task
Output: Table
Description: Creates a Rich Table displaying all tasks

Path: pymc/progress_bar/rich_progress.py
Name: RecolorOnFailureBarColumn

Type: class
Bases: BarColumn
Input: None
Output: None
Description: Progress bar column that changes color on failures

Path: pymc/progress_bar/rich_progress.py
Name: RecolorOnFailureBarColumn._init_

Type: method
Input: self, args, failing_color str, kwargs
Output: None
Description: Initializes with failing color

Path: pymc/progress_bar/rich_progress.py
Name: RecolorOnFailureBarColumn.callbacks

Type: method
Input: self, task Task
Output: Generator
Description: Yields callbacks for bar rendering with dynamic color

Path: pymc/progress_bar/rich_progress.py
Name: RichProgressBackend

Type: class
Implements: ProgressBackend
Input: None
Output: None
Description: Rich-based progress bar backend for terminal rendering

Path: pymc/progress_bar/rich_progress.py

Name: RichProgressBackend._init_

Type: method

Input: self, chains int, total_draws int, combined bool, full_stats bool, progress_columns list, progress_stats dict, theme, disable bool

Output: None

Description: Initializes the Rich progress backend

Path: pymc/progress_bar/rich_progress.py

Name: RichProgressBackend.is_enabled

Type: property

Input: self

Output: bool

Description: Returns True if progress bar is enabled

Path: pymc/progress_bar/rich_progress.py

Name: RichProgressBackend._create_progress_bar

Type: method

Input: self, progress_columns list, theme Theme, disable bool

Output: CustomProgress

Description: Creates the Rich progress bar with appropriate columns

Path: pymc/progress_bar/rich_progress.py

Name: RichProgressBackend._enter_

Type: method

Input: self

Output: RichProgressBackend

Description: Context manager entry

Path: pymc/progress_bar/rich_progress.py

Name: RichProgressBackend._exit_

Type: method

Input: self, exc_type, exc_val, exc_tb

Output: None

Description: Context manager exit

Path: pymc/progress_bar/rich_progress.py

Name: RichProgressBackend._initialize_tasks

Type: method

Input: self

Output: None

Description: Initializes progress bar tasks for all chains

Path: pymc/progress_bar/rich_progress.py

Name: RichProgressBackend.update

Type: method

Input: self, chain_idx int, draw int, failing bool, stats dict, is_last bool

Output: None

Description: Updates progress bar for a specific chain

Path: pymc/progress_bar/marimo_progress.py

Name: load_css

Type: function

Input: css_file str or Path or None

Output: str

Description: Loads CSS and wraps in style tags for HTML embedding

Path: pymc/progress_bar/marimo_progress.py

Name: in_marimo_notebook

Type: function

Input: None

Output: bool

Description: Checks if running inside a marimo notebook

Path: pymc/progress_bar/marimo_progress.py

Name: _mo_write_internal

Type: function

Input: cell_id str, value object

Output: None

Description: Writes to marimo cell given cell_id

Path: pymc/progress_bar/marimo_progress.py

Name: _mo_create_replace

Type: function

Input: None

Output: Callable or None

Description: Creates mo.output.replace with current cell context pinned

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoProgressBackend

Type: class

Implements: ProgressBackend

Input: None

Output: None

Description: Marimo-based progress bar backend for HTML rendering

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoProgressBackend._init_

Type: method

Input: self, chains int, total_draws int, combined bool, full_stats bool, progress_stats dict, css_file str or None

Output: None

Description: Initializes the marimo progress backend

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoProgressBackend.is_enabled

Type: property

Input: self

Output: bool

Description: Returns True always for marimo backend

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoProgressBackend._enter_

Type: method

Input: self

Output: MarimoProgressBackend

Description: Context manager entry that initializes display

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoProgressBackend._exit_

Type: method
Input: self, exc_type, exc_val, exc_tb
Output: None
Description: Context manager exit with final render

Path: pymc/progress_bar/marimo_progress.py
Name: MarimoProgressBackend._initialize_tasks
Type: method
Input: self
Output: None
Description: Initializes progress tracking state for all chains

Path: pymc/progress_bar/marimo_progress.py
Name: MarimoProgressBackend.update
Type: method
Input: self, chain_idx int, draw int, failing bool, stats dict, is_last bool
Output: None
Description: Updates progress for a specific chain

Path: pymc/progress_bar/marimo_progress.py
Name: MarimoProgressBackend._render
Type: method
Input: self
Output: None
Description: Renders the current progress state as HTML

Path: pymc/progress_bar/marimo_progress.py
Name: MarimoProgressBackend._render_html
Type: method
Input: self
Output: str
Description: Generates HTML string for current progress state

Path: pymc/progress_bar/marimo_progress.py
Name: MarimoProgressBackend._render_chain_row
Type: method
Input: self, chain_idx int, state dict, stat_keys list
Output: str
Description: Generates HTML for a single chain row

Path: pymc/progress_bar/marimo_progress.py
Name: MarimoSimpleProgress
Type: class
Input: None
Output: None
Description: Simple progress bar for marimo notebooks

Path: pymc/progress_bar/marimo_progress.py
Name: MarimoSimpleProgress._init_
Type: method
Input: self, description str, total int, disable bool, css_file str or None
Output: None
Description: Initializes simple progress with description and total

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoSimpleProgress._enter_

Type: method

Input: self

Output: MarimoSimpleProgress

Description: Context manager entry

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoSimpleProgress._exit_

Type: method

Input: self, exc_type, exc_val, exc_tb

Output: None

Description: Context manager exit

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoSimpleProgress.add_task

Type: method

Input: self, description str, total int, kwargs

Output: int

Description: Adds a task and returns task ID

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoSimpleProgress.advance

Type: method

Input: self, task_id int or None, advance int

Output: None

Description: Advances progress by specified amount

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoSimpleProgress.update

Type: method

Input: self, task_id int or None, completed int or None, description str or None, kwargs

Output: None

Description: Updates task state

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoSimpleProgress._render

Type: method

Input: self

Output: None

Description: Renders the current progress state

Path: pymc/progress_bar/marimo_progress.py

Name: MarimoSimpleProgress._render_html

Type: method

Input: self

Output: str

Description: Generates HTML string for progress display

Path: pymc/progress_bar/utils.py

Name: compute_draw_speed

Type: function

Input: elapsed float, draws int

Output: tuple of float and str

Description: Computes sampling speed and appropriate unit

Path: pymc/progress_bar/utils.py

Name: format_time

Type: function

Input: seconds float

Output: str

Description: Formats elapsed time as mm ss or hh mm ss

Path: pymc/progress_bar/utils.py

Name: abbreviate_stat_name

Type: function

Input: name str

Output: str

Description: Abbreviates common statistic names for compact display

Path: pymc/progress_bar/progress.css

Name: CSS stylesheet

Type: static asset

Input: None

Output: None

Description: Contains styling for pymc-progress-table and pymc-progress-bar-container and pymc-progress-bar classes with failing and finished states and dark mode support

Multi-Component

The Voila application currently embeds a single widget manager within the core package that only supports ipywidgets 8, breaking backward compatibility for users who have ipywidgets 7 installed. The objective is to decouple widget management from the core Voila package by creating two separate federated JupyterLab extensions that are dynamically selected based on the installed ipywidgets version. Additionally, Voila must introduce its own dedicated labextensions path to prevent polluting the shared JupyterLab environment with Voila-specific extensions.

Create two new TypeScript packages under packages/widgets_manager7/ and packages/widgets_manager8/, each providing the IJupyterWidgetRegistry token and implementing a VoilaWidgetManager class. The widgets-manager7 package must depend on @jupyter-widgets/base ^4.1.6 and @jupyter-widgets/jupyterlab-manager ^3.1.10, with its VoilaWidgetManager extending the WidgetManager class. The widgets-manager8 package must depend on @jupyter-widgets/base ^6.0.10 and @jupyter-widgets/jupyterlab-manager ^5.0.13, with its VoilaWidgetManager extending the KernelWidgetManager class. Both packages must configure their jupyterlab.sharedPackages in package.json with @jupyter-widgets/base and @jupyter-widgets/jupyterlab-manager set to bundled: true and singleton: true to prevent runtime version conflicts. The plugin IDs must be @voila-dashboards/voila:widget-manager7 and @voila-dashboards/voila:widget-manager8 respectively. Both managers must verify they are running in a VoilaApp by checking app.name !== 'Voila', register a widget renderer for the MIME type application/vnd.jupyter.widget-view+json with priority -10, and handle the beforeunload event to send a shutdown beacon. Both must expose registeredModels, a modelRegistered signal, and a removeRegisteredModel() method for progressive rendering support. The widgets-manager7 package must additionally implement a custom VoilaWidgetRenderer class that waits for loadedModelsFromKernel before calling super.renderModel().

Remove the existing widget manager from packages/voila/src/plugins/widget/ entirely, deleting index.ts, plugins.ts, and manager.ts. Relocate the output rendering functionality to a new packages/voila/src/plugins/outputs/ directory by moving renderedcells.ts and tools.ts there. In tools.ts, remove the VoilaWidgetManager import and change the widgetManager parameter type to any. Create a new plugins.ts file containing renderOutputsPlugin and renderOutputsProgressivelyPlugin, ensuring these plugins no longer

require `IJupyterWidgetRegistry` as a dependency. The `renderOutputsPlugin` activation must be wrapped in an `app.started.then()` callback. Update `packages/voila/src/main.ts` to remove the `require('@jupyter-widgets/jupyterlab-manager/lib/plugin')` import and add `'@jupyter-widgets/jupyterlab-manager:plugin'` to the `activePlugins` exclusion list when processing federated extensions. In `packages/voila/src/app.ts`, change the `VoilaWidgetManager` import to a type-only import from `@voila-dashboards/widgets-manager8/lib/manager`. Update `voilaplugins.ts` and `tree.ts` to remove the `widgetManager` references and import from `./plugins/outputs` instead.

Introduce a Voila-specific `labextensions` path at `{PREFIX}/share/jupyter/voila/labextensions`. Create a `get_voila_labextensions_path()` function in `voila/utils.py` that combines standard Jupyter `labextensions` paths from `jupyter_path("labextensions")` with Voila-specific paths derived from `jupyter_path("voila")` by appending `/labextensions` to each path. The function must return standard paths first, followed by Voila-specific paths. Replace all calls to `jupyter_path("labextensions")` in `voila/app.py` and `voila/server_extension.py` with calls to this new function. Remove the conditional logic in `server_extension.py` that previously checked `web_app.settings["labextensions_path"]`.

Implement version detection in a `maybe_inject_widgets_manager_extension()` function in `voila/utils.py`. This function must accept `federated_extensions` and `labextensions_path` parameters, use `get_federated_extensions()` from `jupyterlab_server.config` to check if `@jupyter-widgets/jupyterlab-manager` is installed, and extract its version. Using `packaging.version.Version` for comparison, if the version is `>= 5.0.0`, filter out `@voila-dashboards/widgets-manager7` from the extensions list; otherwise filter out `@voila-dashboards/widgets-manager8`. If the `jupyterlab-manager` is not installed, return the extensions unchanged. Update the `get_page_config()` function to remove `@jupyter-widgets/jupyterlab-manager` from the `disabled_extensions` list and call `maybe_inject_widgets_manager_extension()` after `filter_extension()` but before assigning the result to `page_config["federated_extensions"]`.

Update the directory structure from `voila/labextension` (singular) to `voila/labextensions` (plural) containing subdirectories for each extension: `jupyterlab-preview`, `widgets-manager7`, and `widgets-manager8`. Update `_jupyter_labextension_paths()` in `voila/__init__.py` to return paths for all three extensions, with the widget managers using the `../voila/labextensions/` relative path prefix for their destinations. Update `pyproject.toml` to reflect the new paths in the artifacts list, shared-data mappings (installing widget managers to `share/jupyter/voila/labextensions/`), and `ensured-targets/skip-if-exists` entries. Update `packages/jupyterlab-preview/package.json` to use the new output directory path. Update `.eslintignore`, `.gitignore`, and `.prettierignore` to reference `**/labextensions` instead of `**/labextension`. In `packages/voila/package.json`, remove direct dependencies on `@jupyter-widgets/base` and `@jupyter-widgets/jupyterlab-manager`, add a dependency on `@voila-dashboards/widgets-manager8`, and add dependencies on `@jupyterlab/console`, `@jupyterlab/statedb`, and `@lumino/polling`. Update `sharescope.ts` to import `@jupyterlab/console` and `@jupyterlab/statedb`. Add a `test7` optional dependency group in `pyproject.toml` containing `ipywidgets==7.8.2` for testing backward compatibility.

The implementation introduces the following interfaces. Type: Function. Name: `get_voila_labextensions_path`. Path: `voila/utils.py`. Input: None. Output: `List[str]` (A list of directory paths combining standard Jupyter `labextensions` paths with Voila-specific `labextensions` paths derived from `jupyter_path("voila")`). Description: A utility function that returns all `labextension` search paths for Voila, including both shared Jupyter paths and Voila-specific paths.

Type: Function. Name: `maybe_inject_widgets_manager_extension`. Path: `voila/utils.py`. Input: `federated_extensions: List[Dict]` (List of federated extension configurations.), `labextensions_path: List[str]` (List of `labextension` search paths.). Output: `List[Dict]` (A filtered list of federated extensions with the incompatible widget manager removed based on the installed `ipywidgets` version.). Description: A function that detects the installed `@jupyter-widgets/jupyterlab-manager` version and filters out the incompatible Voila widget manager extension.

Type: Function. Name: `jupyter_labextension_paths`. Path: `voila/init.py`. Input: None. Output: `List[Dict[str, str]]` (A list of dictionaries with "src" and "dest" keys for each labextension.). Description: A Jupyter hook function that returns installation paths for `jupyterlab-preview`, `widgets-manager7`, and `widgets-manager8` extensions.

Type: Class. Name: `VoilaWidgetManager`. Path: `packages/widgets_manager7/src/manager.ts`. Inheritance: extends `WidgetManager` from `@jupyter-widgets/jupyterlab-manager`. Input: context: any (Session context object.), `rendermime`: `IRenderMimeRegistry` (Render MIME registry.), settings: { `saveState`: boolean } (Widget manager settings.). Output: `VoilaWidgetManager` instance. Description: A widget manager class for `ipywidgets 7` support that tracks registered models, emits `modelRegistered` signals, and provides custom `display_view` behavior with `loadedModelsFromKernel` synchronization.

Type: Class. Name: `VoilaWidgetRenderer`. Path: `packages/widgets_manager7/src/index.ts`. Inheritance: extends `WidgetRenderer` from `@jupyter-widgets/jupyterlab-manager`. Input: options: `IRenderMime.IRendererOptions` (Renderer options.), manager: `VoilaWidgetManager` (The Voila widget manager instance.). Output: `VoilaWidgetRenderer` instance. Description: A custom widget renderer that waits for all models to be loaded from the kernel before rendering by awaiting `loadedModelsFromKernel` in the `renderModel` method.

Type: Plugin. Name: `widgetManager`. Path: `packages/widgets_manager7/src/index.ts`. Input: app: `JupyterFrontEnd` (The Voila application.), `rendermime`: `IRenderMimeRegistry` (Render MIME registry.). Output: `IJupyterWidgetRegistry` (The widget registry interface for extension registration.). Description: A JupyterLab extension plugin with ID `@voila-dashboards/voila:widget-manager7` that provides `IJupyterWidgetRegistry` and registers `@jupyter-widgets/controls`, `@jupyter-widgets/base`, and `@jupyter-widgets/output`.

Type: Class. Name: `VoilaWidgetManager`. Path: `packages/widgets_manager8/src/manager.ts`. Inheritance: extends `KernelWidgetManager` from `@jupyter-widgets/jupyterlab-manager`. Input: kernel: `IKernelConnection` (The kernel connection.), `rendermime`: `IRenderMimeRegistry` (Render MIME registry.). Output: `VoilaWidgetManager` instance. Description: A widget manager class for `ipywidgets 8` support that tracks registered models and emits `modelRegistered` signals for progressive rendering coordination.

Type: Plugin. Name: `widgetManager`. Path: `packages/widgets_manager8/src/index.ts`. Input: app: `JupyterFrontEnd` (The Voila application.), `rendermime`: `IRenderMimeRegistry` (Render MIME registry.). Output: `IJupyterWidgetRegistry` (The widget registry interface for extension registration.). Description: A JupyterLab extension plugin with ID `@voila-dashboards/voila:widget-manager8` that provides `IJupyterWidgetRegistry` and uses the standard `WidgetRenderer`.

Type: Plugin. Name: `renderOutputsPlugin`. Path: `packages/voila/src/plugins/outputs/plugins.ts`. Input: app: `JupyterFrontEnd` (The Voila application.), `rendermime`: `IRenderMimeRegistry` (Render MIME registry.). Output: void. Description: A JupyterLab extension plugin with ID `@voila-dashboards/voila:render-outputs` that renders cell outputs and attaches `RenderedCells` widget to the shell on `app.started`.

Type: Plugin. Name: `renderOutputsProgressivelyPlugin`. Path: `packages/voila/src/plugins/outputs/plugins.ts`. Input: app: `JupyterFrontEnd` (The Voila application.), `rendermime`: `IRenderMimeRegistry` (Render MIME registry.). Output: void. Description: A JupyterLab extension plugin with ID `@voila-dashboards/voila:render-outputs-progressively` that handles WebSocket-based progressive rendering when the `progressiveRendering` page config option is enabled.

Full Tool Implementation

Design and implement a C++ header-only library for polygon triangulation that converts 2D polygon vertex data into triangle indices suitable for rendering or geometric processing. The library must accept polygon data where the first ring represents the outer boundary and subsequent rings represent holes to be excluded from triangulation. The implementation should produce a flat vector of indices where every three consecutive indices form a triangle, with vertices numbered sequentially across all input rings. The solution must be entirely self-contained within a single header file (`earcut.hpp`) and depend only on standard C++ library headers,

making it portable and easy to integrate into any C++ project without external dependencies or build system modifications.

The library must support flexible input formats by accepting polygon data as nested vectors where each ring contains coordinate pairs, with built-in support for both `std::array<double, 2>` and `std::pair<double, double>` through a template accessor mechanism (`mapbox::util::nth`). The index type used in output must be configurable via template parameter, defaulting to `uint32_t` for compatibility with common graphics APIs. The namespace structure must clearly separate the public API (`mapbox::earcut`) from internal implementation details (`mapbox::detail::Earcut`), allowing users to call a simple function while hiding algorithmic complexity.

The triangulation algorithm must correctly handle convex polygons, concave polygons, and polygons containing holes by implementing an ear-cutting approach with z-order curve optimization for performance on complex shapes. Holes must be eliminated by computing bridge connections to the outer boundary, effectively converting multi-ring polygons into single-ring equivalents before triangulation. The implementation must include robust geometric primitives for signed area calculation, point-in-triangle testing, segment intersection detection, and collinearity checks, with appropriate handling of floating-point precision edge cases.

Degenerate inputs must be handled gracefully: empty polygon arrays, rings with fewer than three vertices, fully collinear point sequences, and duplicate consecutive vertices must result in empty output without crashes or undefined behavior. The algorithm must cure local self-intersections when detected and employ polygon splitting strategies when simple ear-cutting fails. Memory management must use an object pool pattern to minimize dynamic allocations during processing, with all allocated memory properly released upon completion. The implementation must track vertex counts and support efficient processing of polygons with hundreds or thousands of vertices.

Type: function template

Name: `mapbox::earcut<N, Polygon>`

Path: `earcut.hpp`

Input: `const Polygon& poly` (nested container where `Polygon[0]` is outer ring, `Polygon[1..n]` are holes; each ring contains coordinate pairs accessible via `std::get<0>` and `std::get<1>`)

Output: `std::vector<N>` (flat vector of triangle indices; every 3 consecutive values form one triangle; returns empty vector for degenerate inputs)

Description: Public API entry point that instantiates internal `Earcut<N>` processor, executes triangulation, and returns indices via move semantics.

Type: class template

Name: `mapbox::detail::Earcut<N>`

Path: `earcut.hpp`

Input: N/A (class definition with template parameter `N` for index type)

Output: N/A (members `indices: std::vector<N>` and `vertices: std::size_t` store results)

Description: Internal implementation class containing triangulation algorithm state, node pool, and all geometric operations; not intended for direct use by library consumers.

Type: method template

Name: `mapbox::detail::Earcut<N>::operator()`

Path: `earcut.hpp`

Input: `const Polygon& points` (polygon data as nested vector of rings)

Output: `void` (populates `indices` member with triangle indices and `vertices` with total vertex count)

Description: Main triangulation entry point that resets state, builds linked list from outer ring, eliminates holes by bridging to outer boundary, computes bounding box for z-order hashing optimization when vertex count exceeds threshold of 80, executes ear-cutting loop, and clears node pool on completion.

Type: nested struct

Name: `mapbox::detail::Earcut<N>::Node`

Path: `earcut.hpp`

Input: N index, double x_, double y_ (vertex index and coordinates in constructor)

Output: N/A (data structure with members i, x, y, prev, next, z, prevZ, nextZ, steiner)

Description: Doubly-linked list node representing polygon vertex containing vertex index, coordinates, ring navigation pointers, z-order curve value and pointers for spatial indexing, and steiner point flag; copy and move operations are deleted.

Type: method template

Name: mapbox::detail::Earcut<N>::linkedList<Ring>

Path: earcut.hpp

Input: const Ring& points (single polygon ring), const bool clockwise (desired winding order)

Output: Node* (pointer to last node in created circular doubly-linked list, or nullptr for degenerate rings)

Description: Converts ring coordinates into circular doubly-linked list by calculating winding order via signed area sum and reversing iteration direction if needed to match requested orientation, removes duplicate endpoint if present, and updates global vertex counter.

Type: method

Name: mapbox::detail::Earcut<N>::filterPoints

Path: earcut.hpp

Input: Node* start (starting node), Node* end (optional ending node, defaults to start)

Output: Node* (pointer to filtered list end node)

Description: Eliminates collinear or duplicate points from linked list by removing nodes where area with neighbors equals zero or coordinates match next node, iterating until no more removals possible.

Type: method

Name: mapbox::detail::Earcut<N>::earcutLinked

Path: earcut.hpp

Input: Node* ear (starting node for ear search), int pass (algorithm phase 0-2, default 0)

Output: void (appends triangle indices to indices member)

Description: Core ear-cutting loop where pass 0 attempts direct ear removal with optional z-order indexing, pass 1 filters collinear points and cures local self-intersections, and pass 2 splits polygon via valid diagonal and recurses on both halves; emits three indices to output for each valid ear removed.

Type: method

Name: mapbox::detail::Earcut<N>::isEar

Path: earcut.hpp

Input: Node* ear (candidate ear vertex)

Output: bool (true if vertex forms valid ear with adjacent nodes)

Description: Validates ear candidate by checking for reflex angle via signed area calculation and ensuring no other polygon vertices lie inside the potential triangle formed by the ear and its neighbors.

Type: method

Name: mapbox::detail::Earcut<N>::isEarHashed

Path: earcut.hpp

Input: Node* ear (candidate ear vertex)

Output: bool (true if vertex forms valid ear with adjacent nodes)

Description: Optimized ear validation using z-order curve bounds to limit vertex search range, checking only vertices within the triangle's bounding box z-order range for improved performance on complex polygons.

Type: method

Name: mapbox::detail::Earcut<N>::cureLocalIntersections

Path: earcut.hpp

Input: Node* start (starting node for intersection search)

Output: Node* (pointer to filtered list after intersection removal)

Description: Iterates through polygon nodes detecting and resolving small local self-intersections where edge (v[i-1],v[i]) intersects (v[i+1],v[i+2]), emitting a triangle and removing the two intersecting nodes when found.

Type: method

Name: mapbox::detail::Earcut<N>::splitEarcut

Path: earcut.hpp

Input: Node* start (starting node of polygon to split)

Output: void (recursively triangulates both polygon halves)

Description: Last resort triangulation strategy that searches for valid diagonal dividing polygon into two parts, splits via that diagonal, filters collinear points around cuts, and recursively calls earcutLinked on each half.

Type: method template

Name: mapbox::detail::Earcut<N>::eliminateHoles<Polygon>

Path: earcut.hpp

Input: const Polygon& points (full polygon data with holes), Node* outerNode (outer boundary linked list)

Output: Node* (modified outer node after all holes integrated into single ring)

Description: Processes all holes by creating linked lists for each, finding leftmost vertex of each hole, sorting holes left-to-right by x-coordinate, then bridging each to outer boundary via eliminateHole and filtering collinear points after each bridge insertion.

Type: method

Name: mapbox::detail::Earcut<N>::eliminateHole

Path: earcut.hpp

Input: Node* hole (hole's leftmost vertex), Node* outerNode (outer boundary node)

Output: Node* (updated outer node after hole elimination)

Description: Finds bridge connection point between hole and outer boundary, splits polygon to merge hole into outer ring, filters collinear points around the cuts, and returns appropriate outer node handling the case where input node was removed by filtering.

Type: method

Name: mapbox::detail::Earcut<N>::findHoleBridge

Path: earcut.hpp

Input: Node* hole (leftmost vertex of hole), Node* outerNode (outer boundary starting node)

Output: Node* (optimal bridge connection point on outer boundary, or nullptr if none found)

Description: Implements David Eberly's algorithm by casting horizontal ray leftward from hole vertex, finding intersecting outer edge with endpoint having lesser x as potential connection, then searching for mutually visible vertex within potential triangle using minimum angle tangent criterion.

Type: method

Name: mapbox::detail::Earcut<N>::indexCurve

Path: earcut.hpp

Input: Node* start (starting node of polygon ring)

Output: void (populates z values and links prevZ/nextZ pointers)

Description: Interlinks polygon nodes in z-order by computing z-order curve value for each vertex based on normalized coordinates, setting up z-order linked list pointers, then sorting the list via merge sort for efficient spatial queries.

Type: method

Name: mapbox::detail::Earcut<N>::sortLinked

Path: earcut.hpp

Input: Node* list (head of z-order linked list to sort)

Output: Node* (head of sorted linked list)

Description: Implements Simon Tatham's linked list merge sort algorithm, iteratively merging sublists of increasing size until entire list is sorted by z-order value.

Type: method

Name: mapbox::detail::Earcut<N>::zOrder

Path: earcut.hpp

Input: const double x_ (x coordinate), const double y_ (y coordinate)

Output: int32_t (z-order curve value for spatial indexing)

Description: Computes z-order curve value by transforming coordinates into non-negative 15-bit integer range using bounding box and inverse size, then interleaving bits of x and y coordinates.

Type: method

Name: mapbox::detail::Earcut<N>::getLeftmost

Path: earcut.hpp

Input: Node* start (starting node of polygon ring)

Output: Node* (pointer to leftmost node, using y as tiebreaker)

Description: Finds the leftmost vertex of a polygon ring by iterating through all nodes and tracking minimum x coordinate, with minimum y as secondary criterion for equal x values.

Type: method

Name: mapbox::detail::Earcut<N>::pointInTriangle

Path: earcut.hpp

Input: double ax, double ay, double bx, double by, double cx, double cy, double px, double py (triangle vertices a, b, c and test point p)

Output: bool (true if point lies within or on boundary of triangle)

Description: Tests point containment using three half-plane checks via cross products, where point is considered inside if all three signed area calculations are non-negative.

Type: method

Name: mapbox::detail::Earcut<N>::isValidDiagonal

Path: earcut.hpp

Input: Node* a, Node* b (two polygon vertices to test as diagonal endpoints)

Output: bool (true if diagonal is valid for polygon splitting)

Description: Validates diagonal by checking it doesn't intersect other edges, is locally visible from both endpoints, has middle point inside polygon, doesn't create opposite-facing sectors, with special handling for zero-length case.

Type: method

Name: mapbox::detail::Earcut<N>::area

Path: earcut.hpp

Input: const Node* p, const Node* q, const Node* r (three polygon vertices)

Output: double (signed area of triangle, positive for counter-clockwise, negative for clockwise, zero for collinear)

Description: Computes signed area of triangle using cross product formula $(q.y - p.y) * (r.x - p.x) - (q.x - p.x) * (r.y - p.y)$.

Type: method

Name: mapbox::detail::Earcut<N>::equals

Path: earcut.hpp

Input: const Node* p1, const Node* p2 (two nodes to compare)

Output: bool (true if nodes have identical coordinates)

Description: Tests coordinate equality between two nodes by comparing both x and y values.

Type: method

Name: mapbox::detail::Earcut<N>::intersects

Path: earcut.hpp

Input: const Node* p1, const Node* q1, const Node* p2, const Node* q2 (endpoints of two line segments)

Output: bool (true if segments intersect)

Description: General segment intersection test using orientation via signed area, handling both general case where orientations differ and collinear cases by checking if endpoints lie on the opposite segment.

Type: method

Name: mapbox::detail::Earcut<N>::onSegment

Path: earcut.hpp

Input: const Node* p, const Node* q, const Node* r (three collinear points)

Output: bool (true if q lies on segment pr)

Description: For collinear points, checks if middle point q lies within bounding box defined by segment endpoints p and r.

Type: method

Name: mapbox::detail::Earcut<N>::sign

Path: earcut.hpp

Input: double val (value to get sign of)

Output: int (-1, 0, or 1 indicating sign)

Description: Returns sign of value as integer using expression $(0.0 < \text{val}) - (\text{val} < 0.0)$.

Type: method

Name: mapbox::detail::Earcut<N>::intersectsPolygon

Path: earcut.hpp

Input: const Node* a, const Node* b (diagonal endpoints to test)

Output: bool (true if diagonal intersects any polygon edge)

Description: Checks if proposed diagonal between nodes a and b intersects any existing polygon edge by iterating through all edges and testing intersection, excluding edges sharing endpoints with the diagonal.

Type: method

Name: mapbox::detail::Earcut<N>::locallyInside

Path: earcut.hpp

Input: const Node* a, const Node* b (two polygon vertices)

Output: bool (true if b is locally inside polygon at vertex a)

Description: Checks if vertex b is visible from vertex a considering local polygon geometry, using different criteria based on whether vertex a is convex or reflex.

Type: method

Name: mapbox::detail::Earcut<N>::middleInside

Path: earcut.hpp

Input: const Node* a, const Node* b (diagonal endpoints)

Output: bool (true if midpoint of diagonal lies inside polygon)

Description: Tests if the midpoint of segment ab lies inside polygon using ray casting algorithm, counting edge crossings from midpoint.

Type: method

Name: mapbox::detail::Earcut<N>::splitPolygon

Path: earcut.hpp

Input: Node* a, Node* b (two polygon vertices to connect with bridge)

Output: Node* (pointer to new node b2 in second polygon portion)

Description: Creates bridge between two vertices by allocating duplicate nodes a2 and b2 and relinking pointers; used for hole elimination to merge rings and for polygon splitting to separate rings.

Type: method template

Name: mapbox::detail::Earcut<N>::insertNode<Point>

Path: earcut.hpp

Input: std::size_t i (vertex index), const Point& pt (coordinate pair), Node* last (previous node in list or nullptr)

Output: Node* (pointer to newly created and linked node)

Description: Creates new node from coordinate point and optionally links it into existing circular doubly-linked list after the last node, or creates self-referencing node if last is nullptr.

Type: method

Name: mapbox::detail::Earcut<N>::removeNode

Path: earcut.hpp

Input: Node* p (node to remove from linked list)

Output: void (modifies prev/next and prevZ/nextZ pointers of neighboring nodes)

Description: Removes node from both the polygon ring linked list and the z-order linked list by updating neighbor pointers to skip the removed node.

Type: nested class template

Name: mapbox::detail::Earcut<N>::ObjectPool<T, Alloc>

Path: earcut.hpp

Input: std::size_t blockSize_ (optional allocation block size in constructor)

Output: N/A (provides construct() method returning T* and reset()/clear() for deallocation)

Description: Custom block-based allocator minimizing heap operations by pre-allocating arrays of objects; construct() uses placement new within current block or allocates new block when full; reset() and clear() deallocate all blocks and reset state.

Type: struct template

Name: mapbox::util::nth<I, T>

Path: earcut.hpp

Input: I (std::size_t index into tuple), T (coordinate pair type)

Output: static get() method returns std::tuple_element<I, T>::type

Description: Accessor abstraction enabling uniform coordinate access from std::pair, std::array, std::tuple, or any custom type supporting std::get, allowing polymorphic input format support without requiring users to adapt their data structures.